Bikkad IT

Dream Big! Start Small!!! Act Now!!!

Bikkad IT Institute

Session 5.3: Python Dictionary

Practice Questions with Python Code Answers

Compiled by Bikkad IT Institute | Teaching Notes

WHAT IS INSIDE

1. Create a Dictionary
2. Accessing Items
3. Adding Key-Value Pair
4. Editing Key-Value Pair
5. Dictionary Operations
6. Dictionary Functions
7. Dictionary Comprehension
8. zip() with Dictionaries
9. Nested Comprehensions

54 practice questions | every topic: 2 Beginner + 2 Intermediate + 2 Advanced

BEGINNER INTERMEDIATE ADVANCED

OUR LOCATIONS

+91 77220 82225 Pimpri-Chinchwad

+91 77220 82226 Chh. Sambhajinagar

+91 77220 82227 Nashik

+91 77220 82224 Hadapsar, Pune

Session 5.3: Python Dictionary - Practice Solutions

Prepared for: Bikkad IT Institute Students
Topics: Create a Dictionary, Accessing Items, Adding Key-Value Pair, Editing Key-Value Pair, Dictionary Operations, Dictionary Functions, Dictionary Comprehension, zip() with Dictionaries, Nested Comprehensions
Levels: 2 Beginner + 2 Intermediate + 2 Advanced questions per topic
Date: October 2026

1. Create a Dictionary

Q1. Student Profile

BEGINNER

Create a dictionary for a student with keys name ("Aarav"), age (20), city ("Pune") and course ("Python"). Print it and its type.

Solution:

```
student = {"name": "Aarav", "age": 20,
           "city": "Pune", "course": "Python"}

print(student)
print(type(student))
```

Output:

```
{'name': 'Aarav', 'age': 20, 'city': 'Pune', 'course': 'Python'}
<class 'dict'>
```

Q2. Menu Card

BEGINNER

A cafe menu has Tea 15, Coffee 25, Samosa 20. Store it as a dictionary and print the number of items.

Solution:

```
menu = {"Tea": 15, "Coffee": 25, "Samosa": 20}

print(menu)
print(len(menu))
```

Output:

```
{'Tea': 15, 'Coffee': 25, 'Samosa': 20}
3
```

Q3. Using dict() Function

INTERMEDIATE

Create the same employee record in two ways: with dict(name="Riya", dept="HR") and from a list of pairs [("name", "Riya"), ("dept", "HR")]. Check that both are equal.

Solution:

```
emp1 = dict(name="Riya", dept="HR")
emp2 = dict([("name", "Riya"), ("dept", "HR")])

print(emp1)
print(emp2)
print(emp1 == emp2)
```

Output:

```
{'name': 'Riya', 'dept': 'HR'}
{'name': 'Riya', 'dept': 'HR'}
True
```

Q4. Duplicate Keys

INTERMEDIATE

Create {"Pune": 30, "Goa": 33, "Pune": 32}. Print it and explain in a comment what happened to the duplicate key.

Solution:

```
temps = {"Pune": 30, "Goa": 33, "Pune": 32}

print(temps)
print(len(temps))
# Keys must be unique: the last value (32) replaces the first one
```

Output:

```
{'Pune': 32, 'Goa': 33}
2
```

Q5. Different Key Types

ADVANCED

Create a dictionary using an int key (101), a string key ("code"), and a tuple key ((18.5, 73.8)). Then try a list as key inside try-except.

Solution:

```
info = {101: "Roll", "code": "PY01", (18.5, 73.8): "Pune"}
print(info[(18.5, 73.8)])

try:
    bad = {[1, 2]: "list key"}
except TypeError as e:
    print(e)
```

Output:

```
Pune
unhashable type: 'list'
```

Q6. fromkeys() for Default Values

ADVANCED

Create an attendance dictionary for students ["Om", "Pia", "Raj"] with every value set to "Absent" using dict.fromkeys(). Then mark Pia as "Present".

Solution:

```
students = ["Om", "Pia", "Raj"]

attendance = dict.fromkeys(students, "Absent")
print(attendance)

attendance["Pia"] = "Present"
print(attendance)
```

Output:

```
{'Om': 'Absent', 'Pia': 'Absent', 'Raj': 'Absent'}
{'Om': 'Absent', 'Pia': 'Present', 'Raj': 'Absent'}
```

2. Accessing Items

Q7. Read a Value by Key

BEGINNER

From book = {"title": "Wings of Fire", "author": "A.P.J. Abdul Kalam", "price": 299}, print the title and the price.

Solution:

```
book = {"title": "Wings of Fire",
        "author": "A.P.J. Abdul Kalam",
        "price": 299}

print(book["title"])
print(book["price"])
```

Output:

```
Wings of Fire
299
```

Q8. Safe Access with get()

BEGINNER

Use get() to read "author" and a missing key "pages" from the book dictionary. Also give a default value "Not available" for pages.

Solution:

```
book = {"title": "Wings of Fire",
        "author": "A.P.J. Abdul Kalam",
        "price": 299}

print(book.get("author"))
print(book.get("pages"))
print(book.get("pages", "Not available"))
```

Output:

```
A.P.J. Abdul Kalam
None
Not available
```

Q9. KeyError vs get()

INTERMEDIATE

Access the missing key "discount" in product = {"name": "Mouse", "price": 499} using [] inside try-except and also with get(). Compare the behaviour.

Solution:

```
product = {"name": "Mouse", "price": 499}

try:
    print(product["discount"])
except KeyError as e:
    print("KeyError:", e)

print(product.get("discount", 0))
```

Output:

```
KeyError: 'discount'
0
```

Q10. Loop Through Keys and Values

INTERMEDIATE

Print each item and its stock from stock = {"Rice": 40, "Wheat": 25, "Sugar": 12} using items().

Solution:

```
stock = {"Rice": 40, "Wheat": 25, "Sugar": 12}

for item, qty in stock.items():
    print(item, qty)
```

Output:

```
Rice 40
Wheat 25
Sugar 12
```

Q11. Nested Dictionary Access

ADVANCED

A company dictionary is {"IT": {"head": "Mohan", "staff": 25}, "HR": {"head": "Seema", "staff": 6}}. Print the head of HR and the total staff of all departments.

Solution:

```
company = {
    "IT": {"head": "Mohan", "staff": 25},
    "HR": {"head": "Seema", "staff": 6}
}

print(company["HR"]["head"])

total = 0
for dept in company.values():
    total += dept["staff"]
print(total)
```

Output:

```
Seema
31
```

Q12. Find Key by Value

ADVANCED

Country codes are {"India": "+91", "USA": "+1", "UK": "+44"}. Find the country for the code "+44" by looping over items.

Solution:

```
codes = {"India": "+91", "USA": "+1", "UK": "+44"}
search = "+44"

for country, code in codes.items():
    if code == search:
        print(country)
```

Output:

```
UK
```

3. Adding Key-Value Pair

Q13. Add a Phone Number

BEGINNER

A contact is {"name": "Kunal"}. Add the key "phone" with value "9876543210".

Solution:

```
contact = {"name": "Kunal"}

contact["phone"] = "9876543210"
print(contact)
```

Output:

```
{'name': 'Kunal', 'phone': '9876543210'}
```

Q14. Add New Menu Item

BEGINNER

Add "Vada Pav" with price 20 to the menu {"Tea": 15, "Coffee": 25} and print the number of items.

Solution:

```
menu = {"Tea": 15, "Coffee": 25}

menu["Vada Pav"] = 20
print(menu)
print(len(menu))
```

Output:

```
{'Tea': 15, 'Coffee': 25, 'Vada Pav': 20}
3
```

Q15. Add Many with update()

INTERMEDIATE

Add {"email": "kunal@mail.com", "city": "Nashik"} to the contact {"name": "Kunal"} using update().

Solution:

```
contact = {"name": "Kunal"}

contact.update({"email": "kunal@mail.com", "city": "Nashik"})
print(contact)
```

Output:

```
{'name': 'Kunal', 'email': 'kunal@mail.com', 'city': 'Nashik'}
```

Q16. Add Only If Missing

INTERMEDIATE

Use setdefault() to add "country": "India" to user = {"name": "Ritu", "country": "Nepal"} and "language": "Hindi". Observe which one is actually added.

Solution:

```
user = {"name": "Ritu", "country": "Nepal"}

user.setdefault("country", "India")
user.setdefault("language", "Hindi")
print(user)
```

Output:

```
{'name': 'Ritu', 'country': 'Nepal', 'language': 'Hindi'}
```

Q17. Word Frequency Counter

ADVANCED

Count how many times each word appears in "to be or not to be" by adding keys to an empty dictionary.

Solution:

```
text = "to be or not to be"
freq = {}

for word in text.split():
    if word in freq:
        freq[word] += 1
    else:
        freq[word] = 1

print(freq)
```

Output:

```
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
```

Q18. Group Students by City

ADVANCED

Students are [("Om", "Pune"), ("Pia", "Goa"), ("Raj", "Pune"), ("Sam", "Goa"), ("Tia", "Agra")]. Build a dictionary {city: [names]} using setdefault().

Solution:

```
students = [("Om", "Pune"), ("Pia", "Goa"), ("Raj", "Pune"),
            ("Sam", "Goa"), ("Tia", "Agra")]
by_city = {}

for name, city in students:
    by_city.setdefault(city, []).append(name)

print(by_city)
```

Output:

```
{'Pune': ['Om', 'Raj'], 'Goa': ['Pia', 'Sam'], 'Agra': ['Tia']}
```

4. Editing Key-Value Pair

Q19. Update a Price

BEGINNER

In menu = {"Tea": 15, "Coffee": 25}, change the price of Tea to 18.

Solution:

```
menu = {"Tea": 15, "Coffee": 25}

menu["Tea"] = 18
print(menu)
```

Output:

```
{'Tea': 18, 'Coffee': 25}
```

Q20. Increase Stock

BEGINNER

Stock is {"Pens": 50, "Books": 20}. 30 new pens arrive. Increase the pen stock using +=.

Solution:

```
stock = {"Pens": 50, "Books": 20}

stock["Pens"] += 30
print(stock)
```

Output:

```
{'Pens': 80, 'Books': 20}
```

Q21. Update Several Values

INTERMEDIATE

Employee record {"name": "Ajay", "dept": "Sales", "salary": 40000}. After promotion, update dept to "Marketing" and salary to 52000 in one update() call.

Solution:

```
emp = {"name": "Ajay", "dept": "Sales", "salary": 40000}

emp.update(dept="Marketing", salary=52000)
print(emp)
```

Output:

```
{'name': 'Ajay', 'dept': 'Marketing', 'salary': 52000}
```

Q22. 10% Hike for Everyone

INTERMEDIATE

Salaries are {"Om": 30000, "Pia": 45000, "Raj": 38000}. Increase every salary by 10% using a loop over keys.

Solution:

```
salaries = {"Om": 30000, "Pia": 45000, "Raj": 38000}

for name in salaries:
    salaries[name] = int(salaries[name] * 1.10)

print(salaries)
```

Output:

```
{'Om': 33000, 'Pia': 49500, 'Raj': 41800}
```

Q23. Rename a Key

ADVANCED

Rename the key "mob" to "mobile" in user = {"name": "Isha", "mob": "9811122233"} using pop().

Solution:

```
user = {"name": "Isha", "mob": "9811122233"}

user["mobile"] = user.pop("mob")
print(user)
```

Output:

```
{'name': 'Isha', 'mobile': '9811122233'}
```

Q24. Edit Nested Data

ADVANCED

Cart is {"items": {"Shoes": 1, "Socks": 2}, "coupon": None}. Increase Socks to 3, add "Cap": 1 and set coupon to "SAVE10".

Solution:

```
cart = {"items": {"Shoes": 1, "Socks": 2}, "coupon": None}

cart["items"]["Socks"] = 3
cart["items"]["Cap"] = 1
cart["coupon"] = "SAVE10"
print(cart)
```

Output:

```
{'items': {'Shoes': 1, 'Socks': 3, 'Cap': 1}, 'coupon':
'SAVE10'}
```

5. Dictionary Operations

Q25. Membership Works on Keys

BEGINNER

For scores = {"Om": 80, "Pia": 92}, check "Om" in scores and 92 in scores. Then check 92 in scores.values().

Solution:

```
scores = {"Om": 80, "Pia": 92}

print("Om" in scores)
print(92 in scores)
print(92 in scores.values())
```

Output:

```
True
False
True
```

Q26. Delete with del

BEGINNER

Remove the key "Coffee" from menu = {"Tea": 15, "Coffee": 25, "Milk": 20} using del.

Solution:

```
menu = {"Tea": 15, "Coffee": 25, "Milk": 20}

del menu["Coffee"]
print(menu)
```

Output:

```
{'Tea': 15, 'Milk': 20}
```

Q27. Merge Two Dictionaries

INTERMEDIATE

Merge price lists shop1 = {"Pen": 10, "Book": 60} and shop2 = {"Book": 55, "Bag": 450} using the | operator (Python 3.9+). Which Book price is kept?

Solution:

```
shop1 = {"Pen": 10, "Book": 60}
shop2 = {"Book": 55, "Bag": 450}

merged = shop1 | shop2
print(merged)
```

Output:

```
{'Pen': 10, 'Book': 55, 'Bag': 450}
```

Q28. Loop with Condition

INTERMEDIATE

Print only the products with price above 100 from {"Pen": 10, "Bag": 450, "Book": 60, "Shoes": 1200}.

Solution:

```
prices = {"Pen": 10, "Bag": 450, "Book": 60, "Shoes": 1200}

for item, price in prices.items():
    if price > 100:
        print(item, price)
```

Output:

```
Bag 450
Shoes 1200
```

Q29. Compare Two Dictionaries

ADVANCED

Check whether two settings dictionaries are equal even though keys are in a different order: {"theme": "dark", "size": 14} and {"size": 14, "theme": "dark"}.

Solution:

```
a = {"theme": "dark", "size": 14}
b = {"size": 14, "theme": "dark"}

print(a == b)
print(a is b)
```

Output:

```
True
False
```

Q30. Combine Sales from Two Days

ADVANCED

Day 1 sales {"Tea": 40, "Coffee": 25} and Day 2 sales {"Tea": 35, "Juice": 10}. Create total sales by adding values of common items.

Solution:

```
day1 = {"Tea": 40, "Coffee": 25}
day2 = {"Tea": 35, "Juice": 10}

total = day1.copy()
for item, qty in day2.items():
    total[item] = total.get(item, 0) + qty

print(total)
```

Output:

```
{'Tea': 75, 'Coffee': 25, 'Juice': 10}
```

6. Dictionary Functions

Q31. keys(), values(), items()

BEGINNER

For fees = {"Python": 15000, "SQL": 8000, "Excel": 5000} print the keys, the values and the items as lists.

Solution:

```
fees = {"Python": 15000, "SQL": 8000, "Excel": 5000}

print(list(fees.keys()))
print(list(fees.values()))
print(list(fees.items()))
```

Output:

```
['Python', 'SQL', 'Excel']
[15000, 8000, 5000]
[('Python', 15000), ('SQL', 8000), ('Excel', 5000)]
```

Q32. len, sum, max of Values

BEGINNER

For the same fees dictionary, print the number of courses, total fees, and the maximum fee.

Solution:

```
fees = {"Python": 15000, "SQL": 8000, "Excel": 5000}

print(len(fees))
print(sum(fees.values()))
print(max(fees.values()))
```

Output:

```
3
28000
15000
```

Q33. pop() and popitem()

INTERMEDIATE

From cart = {"Shoes": 1, "Cap": 2, "Belt": 1} remove "Cap" using pop() (print the removed value). Then remove the last inserted item using popitem().

Solution:

```
cart = {"Shoes": 1, "Cap": 2, "Belt": 1}

removed = cart.pop("Cap")
print(removed)
print(cart)

last = cart.popitem()
print(last)
print(cart)
```

Output:

```
2
{'Shoes': 1, 'Belt': 1}
('Belt', 1)
{'Shoes': 1}
```

Q34. copy() and clear()

INTERMEDIATE

Make a backup of settings = {"wifi": True, "bt": False} using copy(), then clear() the original. Print both.

Solution:

```
settings = {"wifi": True, "bt": False}

backup = settings.copy()
settings.clear()

print(settings)
print(backup)
```

Output:

```
{}
{'wifi': True, 'bt': False}
```

Q35. Course with Highest Fee

ADVANCED

Find the course with the highest fee and the course with the lowest fee from {"Python": 15000, "SQL": 8000, "Excel": 5000} using max()/min() with key=fees.get.

Solution:

```
fees = {"Python": 15000, "SQL": 8000, "Excel": 5000}

print(max(fees, key=fees.get))
print(min(fees, key=fees.get))
```

Output:

```
Python
Excel
```

Q36. Sort a Dictionary by Value

ADVANCED

Sort the scores {"Om": 72, "Pia": 95, "Raj": 81} from highest to lowest and store the result in a new dictionary.

Solution:

```
scores = {"Om": 72, "Pia": 95, "Raj": 81}

def by_score(pair):
    return pair[1]

ranked = dict(sorted(scores.items(), key=by_score, reverse=True))
print(ranked)
```

Output:

```
{'Pia': 95, 'Raj': 81, 'Om': 72}
```

7. Dictionary Comprehension

Q37. Squares Table

BEGINNER

Create {1: 1, 2: 4, ..., 5: 25} using dictionary comprehension.

Solution:

```
squares = {n: n * n for n in range(1, 6)}
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Q38. Word Lengths

BEGINNER

Create a dictionary of each fruit and its length from ["mango", "kiwi", "banana"].

Solution:

```
fruits = ["mango", "kiwi", "banana"]

lengths = {f: len(f) for f in fruits}
print(lengths)
```

Output:

```
{'mango': 5, 'kiwi': 4, 'banana': 6}
```

Q39. Apply Discount

INTERMEDIATE

Prices are {"Shirt": 800, "Jeans": 1500, "Cap": 300}. Create a new dictionary with a 20% discount applied.

Solution:

```
prices = {"Shirt": 800, "Jeans": 1500, "Cap": 300}

sale = {item: price * 0.8 for item, price in prices.items()}
print(sale)
```

Output:

```
{'Shirt': 640.0, 'Jeans': 1200.0, 'Cap': 240.0}
```

Q40. Filter Passed Students

INTERMEDIATE

From marks = {"Om": 32, "Pia": 78, "Raj": 45, "Sam": 28}, keep only students with marks >= 35.

Solution:

```
marks = {"Om": 32, "Pia": 78, "Raj": 45, "Sam": 28}

passed = {n: m for n, m in marks.items() if m >= 35}
print(passed)
```

Output:

```
{'Pia': 78, 'Raj': 45}
```

Q41. Grade Labels with if-else

ADVANCED

Convert marks {"Om": 32, "Pia": 78, "Raj": 45} into {name: "Pass"/"Fail"} using if-else inside the comprehension.

Solution:

```
marks = {"Om": 32, "Pia": 78, "Raj": 45}

result = {n: ("Pass" if m >= 35 else "Fail")
          for n, m in marks.items()}
print(result)
```

Output:

```
{'Om': 'Fail', 'Pia': 'Pass', 'Raj': 'Pass'}
```

Q42. Swap Keys and Values

ADVANCED

Invert the dictionary {"IN": "India", "JP": "Japan", "FR": "France"} so country names become keys.

Solution:

```
codes = {"IN": "India", "JP": "Japan", "FR": "France"}

inverted = {country: code for code, country in codes.items()}
print(inverted)
```

Output:

```
{'India': 'IN', 'Japan': 'JP', 'France': 'FR'}
```

8. zip() with Dictionaries

Q43. Build a Dictionary from Two Lists

BEGINNER

Subjects ["Maths", "Science", "English"] and marks [88, 76, 91]. Create a dictionary using dict(zip()).

Solution:

```
subjects = ["Maths", "Science", "English"]
marks = [88, 76, 91]

report = dict(zip(subjects, marks))
print(report)
```

Output:

```
{'Maths': 88, 'Science': 76, 'English': 91}
```

Q44. Phone Book

BEGINNER

Names ["Asha", "Ravi"] and numbers ["98111", "98222"]. Build a phone book with zip and print Ravi's number.

Solution:

```
names = ["Asha", "Ravi"]
numbers = ["98111", "98222"]

phone_book = dict(zip(names, numbers))
print(phone_book["Ravi"])
```

Output:

```
98222
```

Q45. Comprehension with zip

INTERMEDIATE

Items ["Pen", "Book", "Bag"], quantities [3, 2, 1], rates [10, 60, 450]. Build {item: amount} using zip inside a dictionary comprehension.

Solution:

```
items = ["Pen", "Book", "Bag"]
qty = [3, 2, 1]
rate = [10, 60, 450]

bill = {i: q * r for i, q, r in zip(items, qty, rate)}
print(bill)
print(sum(bill.values()))
```

Output:

```
{'Pen': 30, 'Book': 120, 'Bag': 450}
600
```

Q46. Split a Dictionary into Lists

INTERMEDIATE

Split prices = {"Tea": 15, "Coffee": 25} back into a tuple of names and a tuple of prices using zip(*items()).

Solution:

```
prices = {"Tea": 15, "Coffee": 25}

names, values = zip(*prices.items())
print(names)
print(values)
```

Output:

```
('Tea', 'Coffee')
(15, 25)
```

Q47. Rows from a CSV Header

ADVANCED

A CSV header is ["id", "name", "city"] and rows are [[1, "Om", "Pune"], [2, "Pia", "Goa"]]. Convert every row into a dictionary using zip.

Solution:

```
header = ["id", "name", "city"]
rows = [[1, "Om", "Pune"], [2, "Pia", "Goa"]]

records = [dict(zip(header, row)) for row in rows]
for r in records:
    print(r)
```

Output:

```
{'id': 1, 'name': 'Om', 'city': 'Pune'}
{'id': 2, 'name': 'Pia', 'city': 'Goa'}
```

Q48. Compare Expected vs Actual

ADVANCED

Expected stock {"Rice": 50, "Dal": 30, "Oil": 20} and actual counts in the same order [48, 30, 15]. Use zip to print items where the count does not match and by how much.

Solution:

```
expected = {"Rice": 50, "Dal": 30, "Oil": 20}
actual = [48, 30, 15]

for (item, exp), act in zip(expected.items(), actual):
    if exp != act:
        print(item, exp - act)
```

Output:

```
Rice 2
Oil 5
```

9. Nested Comprehensions

Q49. Multiplication Tables

BEGINNER

Create a dictionary where keys are 2, 3, 4 and values are lists of the first 5 multiples, using a nested comprehension.

Solution:

```
tables = {n: [n * i for i in range(1, 6)] for n in range(2, 5)}
print(tables)
```

Output:

```
{2: [2, 4, 6, 8, 10], 3: [3, 6, 9, 12, 15], 4: [4, 8, 12, 16,
20]}
```

Q50. Letter Counts per Word

BEGINNER

For ["pune", "goa"], create {word: {letter: count}} using a nested comprehension.

Solution:

```
words = ["pune", "goa"]

result = {w: {ch: w.count(ch) for ch in w} for w in words}
print(result)
```

Output:

```
{'pune': {'p': 1, 'u': 1, 'n': 1, 'e': 1}, 'goa': {'g': 1, 'o':
1, 'a': 1}}
```

Q51. Student Totals

INTERMEDIATE

Marks are {"Om": {"Maths": 80, "Sci": 70}, "Pia": {"Maths": 95, "Sci": 88}}. Create {name: total} using comprehension.

Solution:

```
marks = {"Om": {"Maths": 80, "Sci": 70},
         "Pia": {"Maths": 95, "Sci": 88}}

totals = {name: sum(subj.values())
          for name, subj in marks.items()}
print(totals)
```

Output:

```
{'Om': 150, 'Pia': 183}
```

Q52. Apply GST to Nested Prices

INTERMEDIATE

Prices by category are {"Food": {"Pizza": 200}, "Electronics": {"Mouse": 500}}. Add 18% GST to every price using a nested dictionary comprehension.

Solution:

```
prices = {"Food": {"Pizza": 200}, "Electronics": {"Mouse": 500}}

with_gst = {cat: {item: p * 1.18 for item, p in items.items()}
            for cat, items in prices.items()}
print(with_gst)
```

Output:

```
{'Food': {'Pizza': 236.0}, 'Electronics': {'Mouse': 590.0}}
```

Q53. Filter Inside Nested Data

ADVANCED

From the same marks dictionary, keep only subjects where the mark is at least 80 for each student.

Solution:

```
marks = {"Om": {"Maths": 80, "Sci": 70},
         "Pia": {"Maths": 95, "Sci": 88}}

good = {name: {s: m for s, m in subj.items() if m >= 80}
        for name, subj in marks.items()}
print(good)
```

Output:

```
{'Om': {'Maths': 80}, 'Pia': {'Maths': 95, 'Sci': 88}}
```

Q54. Seating Chart

ADVANCED

Create a seating chart dictionary for rows "A", "B", "C" where each row maps to a dictionary {seat_no: "Empty"} for seats 1 to 3. Then book seat B2.

Solution:

```
chart = {row: {seat: "Empty" for seat in range(1, 4)}
         for row in "ABC"}

chart["B"][2] = "Booked"
for row, seats in chart.items():
    print(row, seats)
```

Output:

```
A {1: 'Empty', 2: 'Empty', 3: 'Empty'}
B {1: 'Empty', 2: 'Booked', 3: 'Empty'}
C {1: 'Empty', 2: 'Empty', 3: 'Empty'}
```

End of Solutions

Prepared by: Bikkad IT Institute
For: Python Beginner Level Students
Date: October 2026